God's number is at least 20: proving it in Rocq

A guide to the Rubik's cube development, for readers who do not read Rocq

Laurent Théry

INRIA, Stamp Team
Laurent.Thery@inria.fr

Abstract. God's number, the largest number of face turns needed to solve a Rubik's cube, is twenty. This note describes a proof in the Rocq prover of the lower half of that statement: one position, the superflip, cannot be solved in nineteen moves. It presents the cube as a group of permutations of its forty-eight stickers, the pruned search that establishes the bound, the two conditions a pruning table must satisfy, and only those, so that a table with two billion entries never has to be proved correct, and the work needed to make a kernel run such a search at all: machine integers instead of unary ones, guards that do not evaluate what they guard, a search proved equal to a twelve times faster one, and a table folded by symmetry. The final theorem is admit-free; the search behind it took 85 hours of processor time.

Keywords. Rubik's cube, God's number, formal proof, Rocq, mathcomp, pruning table, heuristic search.

1 The problem

A Rubik's cube is built from twenty-six small cubes: **eight corner pieces** with three stickers each, **twelve edge pieces** with two, and **six centre pieces** with one. The centres are attached to the core. They spin in place but never travel, so they are the frame that everything else is measured against: the white face is wherever the white centre is. A face turn moves four corners and four edges, and nothing else.

Not every arrangement of the pieces can be reached by turning faces. Those that can number

$$8! \cdot 3^7 \cdot 12! \cdot 2^{11}/2 = 43\ 252\ 003\ 274\ 489\ 856\ 000 \approx 4.3 \cdot 10^{19},$$

which reads: the eight corners in any order ($8!$), each twisted one of three ways except that the last is forced by the other seven (3^7); the twelve edges in any order ($12!$), each flipped or not with the last again forced (2^{11}); and a final halving, because corners and edges cannot be rearranged independently of each other.

Three details of the pieces will matter later, and they are worth naming now. Every corner carries exactly one sticker of the top or bottom colour, and that sticker sits either on the top of the corner or on one of its two sides: which of the three is the corner's **twist**. Every edge has a right way round, and can sit in its slot **turned over**, showing its two colours the other way about. That is what the superflip does, to all twelve edges at once. And the four edges lying in the **middle layer**, the slice between the top and bottom faces, occupy four of the twelve edge slots; which four is the third thing to keep track of.

Turning one face is a *move*, and a half turn counts as one move just like a quarter turn. Every scramble can be undone; the question is how many moves the worst scramble needs. That number is called **God's number**.

In 2010 Rokicki, Kociemba, Davidson and Dethridge showed that it is **20** [1]. The answer comes in two halves, and they are not equally hard.

- **Twenty moves are always enough.** This is the huge half: every one of the 43 quintillion states has to be accounted for. It took about a billion seconds of processor time, donated by Google, after the states had been grouped into 55 882 296 families.
- **Twenty moves are sometimes needed.** For this it is enough to point at one scramble and show it cannot be solved in 19.

This note is about the second half, and about one scramble: the **superflip**, the cube in which every corner is already correct and all twelve edges are flipped in place (Figure 1). A 20-move solution for it is known. What has to be proved is that no solution of 19 moves exists.

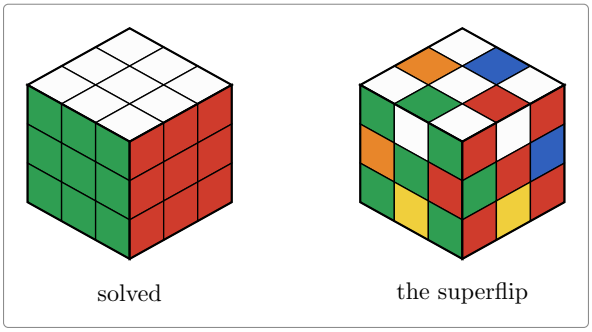

Figure 1: The position the proof is about. Every corner sticker is where it belongs; every edge cubie is in its right place but turned over, so it shows the colour of the face beside it. Turn the whole cube in your hands, or look at it in a mirror, and the same pattern comes back: the superflip is one of the rare positions left unchanged by all 48 ways of looking at a cube. That is what lets the search fix its first move.

That is still a big computation. There are 18 possible moves at each step, so 19 moves means something like 18^{19} words, far more than could ever be listed. Nobody searches like that: the search is cut short using a precomputed table, and that is where both the interest and the difficulty lie.

Why do it in a proof assistant at all, here the Rocq prover [2] (the system formerly called Coq)? Because then the result no longer depends on a search program being right. What is checked instead is a proof, verified by a small kernel, starting from the definition of the cube itself.

2 The cube as permutations

The cube is easier to reason about if we stop thinking about it as a solid object. Only the coloured stickers matter. There are six faces of nine stickers, and the six centre stickers never move relative to each other, so a move is just a rearrangement of the **48 remaining stickers**. Number them 0 to 47, as in Figure 2 and Figure 3.

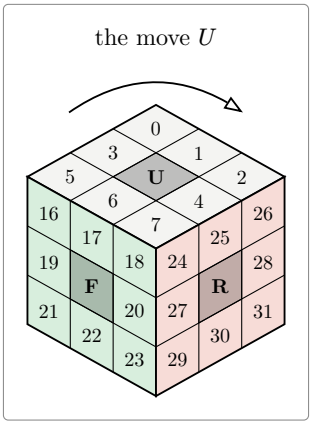

Figure 2: Three of the six faces, with the numbering the development uses. Hidden behind them are the left face (8–15), the back (32–39) and the bottom (40–47). The six centre stickers, marked with a letter, never move.

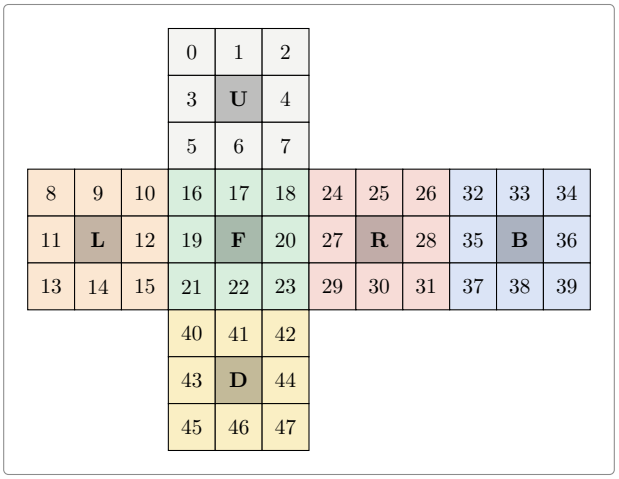

Figure 3: The cube unfolded. Each face is read left to right, top to bottom, with its centre skipped: up 0–7, left 8–15, front 16–23, right 24–31, back 32–39, down 40–47. These are the numbers the Rocq file uses, so the definition of a move can be checked against this picture.

With the places numbered, a move is written down by saying, for each place, where the sticker sitting there goes. Turning the top face clockwise carries the sticker in corner 0 to corner 2, the one in 2 to 7, the one in 7 to 5 and the one in 5 back to 0. That is a four step cycle, written (0 2 7 5). The four edge stickers of that face do the same, (1 4 6 3). But the turn does not only move the top face: it also carries the top row of each side face round to the next one, front to left, left to back, back to right, right to front. That is three more cycles, (8 32 24 16) and its two companions, and Figure 4 shows the whole of it.

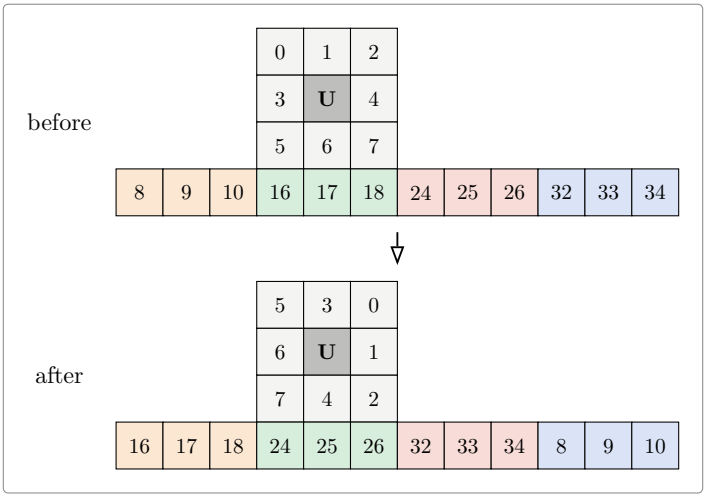

Figure 4: Everything a clockwise turn of the top face moves: the nine places of that face, and the top row of the left, front, right and back faces below it. Each square says which sticker sits there, so the top row of the left face now holds 16, 17, 18, the stickers that came round from the front. The other forty stickers are untouched, and the six centres never move at all.

Six clockwise quarter turns generate everything: up, right, front, down, left and back. Each of them can also be done twice or backwards, which gives the **eighteen moves**

$$U,\ U^2,\ U^{-1},\quad R,\ R^2,\ R^{-1},\quad F,\ F^2,\ F^{-1},\quad D,\ D^2,\ D^{-1},\quad L,\ L^2,\ L^{-1},\quad B,\ B^2,\ B^{-1}$$

A scramble is a product of moves, for instance $RUR^{-1}U^{-1}$, a word of length 4. The set of all scrambles is a group G: the **cube group**. Solving a scramble in d moves means writing it as a word of d moves, so "solvable in at most d moves" says exactly that the scramble lies in the **ball of radius d** around the solved cube. God's number is the largest distance that occurs, the diameter of that ball structure.

2.1 What this looks like in Rocq

The development is built on **mathcomp** [3], a large library of formalised mathematics that already knows about permutations, groups and products. The cube file is then just a transcription of the paragraphs above, and it is short:

```
Definition facelet := 'I_48.

Definition Umove : {perm facelet} :=
  cyc [:: 0@; 2@; 7@; 5@] * cyc [:: 1@; 4@; 6@; 3@] *
  cyc [:: 8@; 32@; 24@; 16@] * cyc [:: 9@; 33@; 25@; 17@] *
  cyc [:: 10@; 34@; 26@; 18@].
  (* ... and five more, one per face ... *)

Definition faces : seq {perm facelet} :=
  [:: Umove; Rmove; Fmove; Dmove; Lmove; Bmove].
Definition moves : seq {perm facelet} :=
  flatten [seq [:: g; g ^+ 2; g ^-1] | g <- faces].
Definition G : {group {perm facelet}} := <<Sset>>.
```

Word by word:

- `'I_48` is the type of the whole numbers **below** 48, so the places are numbered **0 to 47** and not 1 to 48, everywhere in the sources and in the pictures of this note.
- `{perm facelet}` is the type of **permutations** of those places: a way of sending each place to a place, no two of them landing on the same one. That is exactly what a scramble is.
- `cyc [:: 0@; 2@; 7@; 5@]` is the **cycle** that sends 0 to 2, 2 to 7, 7 to 5 and 5 back to 0, leaving the other forty-four places where they are. The `@` is local notation turning a plain number into a place.
- `*` composes two permutations, so `Umove` is the five cycles of Figure 4 done together, and `g ^+ 2` and `g ^-1` are the same turn done twice and undone.
- `seq` is a list, and `faces` is the list of the six clockwise quarter turns. `moves` runs through it and keeps three moves per face, which is the eighteen.
- `<<Sset>>` is the group generated by a set: everything reachable by composing moves, which is the cube group.

The superflip is defined the same way, as the twelve swaps that exchange the two stickers of each edge. Two facts about it are then proved, and both are ordinary algebra rather than computation. Applying the superflip twice returns the cube to solved, since every edge is turned over and then turned over again. And the superflip is the result of the 20-move sequence

$$U\ R^2\ F\ B\ R\ B^2\ R\ U^2\ L\ B^2\ R\ U^{-1}\ D^{-1}\ R^2\ F\ R^{-1}\ L\ B^2\ U^2\ F^2$$

which is what makes it a legal scramble, and gives the matching upper bound of 20 for this particular cube.

Nothing here is assumed. There is no axiom stating what a cube is, and a reader who wants to check the model only has to compare the six lists of cycles against Figure 3. After this file, stickers are never mentioned again.

3 How the search works

Two classical ideas make the search possible.

3.1 An estimate that is never too big

The idea is not new, and neither is the way the estimate is obtained. A depth-first search that deepens step by step and prunes on an estimate that never over-estimates is Korf's IDA* [4]; taking the estimate from a table of exact distances in a simplified version of the puzzle is a *pattern database* [5],

and Korf solved the cube optimally with three of them [6]. The summary used here is Kociemba's, from his two-phase solver [7].

Suppose we have a way of estimating, for any scramble, how many moves it needs : an estimate that is never larger than the truth, and that changes by at most one when a move is made. Call it h.

Such an estimate is a pair of scissors. Walk down the tree of moves, keeping track of how many are left. If the estimate for a position is 20 while only 18 moves remain, that whole branch can be dropped: it cannot possibly reach the solved cube in time. Figure 5 shows the picture.

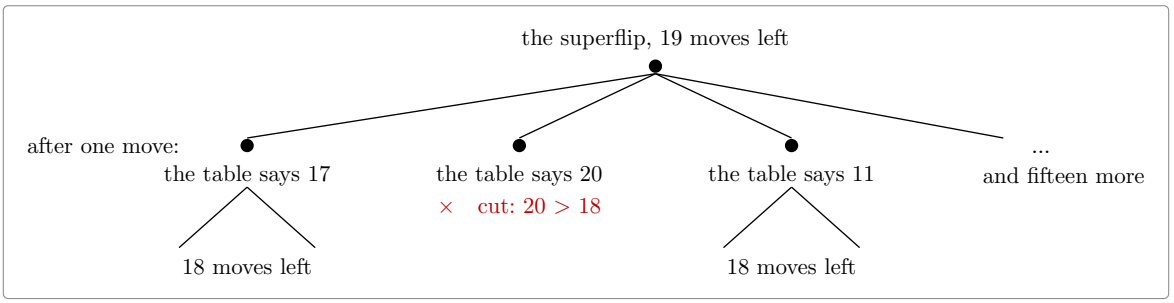

Figure 5: The search, and its scissors. Below every position the table is consulted; a branch whose estimate exceeds the moves still available is abandoned without ever being explored. The estimate is allowed to be too small, which only means less cutting, but never too large.

3.2 Where the estimate comes from

The trick is to forget most of the cube. Keep only part of the information, say how the corners are twisted and where the four middle-layer edges sit, and call what is left a **summary**. Many different scrambles share a summary. Moves act on summaries just as well as on cubes, and there are few enough summaries that a computer can work out, once and for all, the exact distance from the solved summary to every other summary. That table of distances is the estimate: a scramble needs at least as many moves as its summary does.

And here is where the facelet numbering earns its keep: a summary is read straight off the stickers. Figure 6 shows the three questions the summary asks.

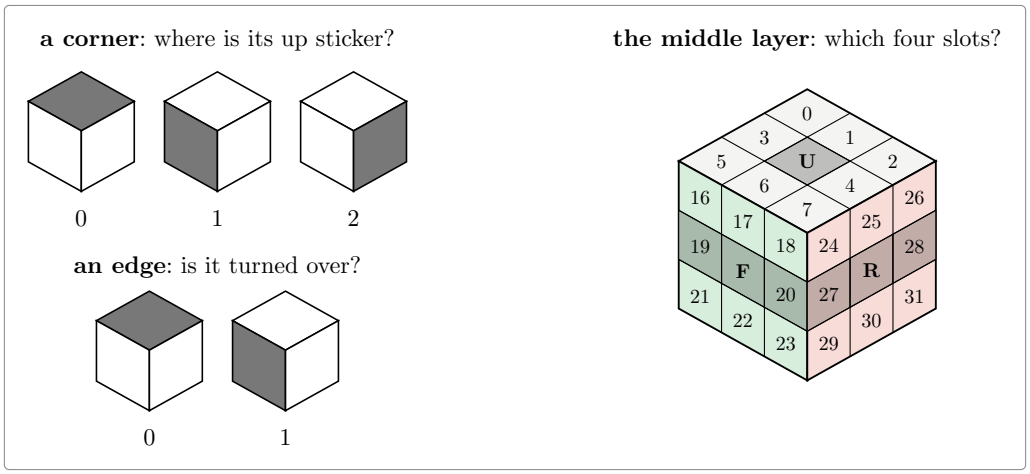

Figure 6: The three questions. A corner has one sticker belonging to the up or down face, and it sits in one of three places: 0, 1 or 2. An edge is either the right way round or turned over: 0 or 1. And the four edges of the middle layer occupy four of the twelve edge slots, shaded here on the two visible faces, where three of the four can be seen. Nothing else about the position is recorded.

The summary is the product of the three answers:

summary	values	
how the eight corners are twisted	2 187	$= 3^7$
how the twelve edges are flipped	2 048	$= 2^{11}$
where the four middle-layer edges sit	495	4 places among 12
the phase 1 summary, all three together	**2 217 093 120**	

Two billion is small next to 43 quintillion: **every summary stands for exactly 19 508 428 800 real scrambles** (that is the division, and it comes out even). The table records, for each summary, its distance from the solved summary. No entry in it is larger than 12, that being how deep the deepest summary lies rather than the value of any position in particular. So four bits hold one entry, and the whole table is **1.18 GB**.

3.3 And why a failed search is a proof

The search answers "maybe solvable in d moves" or "no". The **no** is the trustworthy side: it means every branch was cut, either because the depth ran out or because the table said so. So a negative answer to "is the superflip solvable in 19 moves?" is exactly the theorem wanted.

Better still, the answer does not depend on the table being right. If an entry of the table were too small, the search would merely cut less and take longer. Only the two local properties matter: the solved summary has distance 0, and one move changes the value by at most one. That is what makes the whole thing formalisable at a sane cost: a table with two billion entries never has to be proved correct, only checked.

4 The search in Rocq

The general principle is one file of about a hundred lines, `Search.v`, which does not mention the cube at all. It takes a group, a set of moves, an estimate h, and the two assumptions:

```
Hypothesis h1    : h 1 = 0.
Hypothesis hstep : forall g m, m \in S -> h g <= (h (g * m)).+1.

Fixpoint search (d : nat) (g : gT) : bool :=
  (h g <= d) &&
  ((g == 1) ||
   (if d is d'.+1 then has (fun m => search d' (g * m)) Sseq else false)).

Corollary searchN d g : search d g = false -> g \notin ball S d.
```

Again word by word:

- `gT` is the group the file works in. It is a variable, so nothing here is about the cube; the cube is what it gets instantiated with later.
- `1` is the unit of that group, which for the cube is the **solved** position. So `h 1 = 0` says the estimate of a solved cube is zero, and `g == 1` asks whether the search has arrived.
- `Sseq` is the list of moves, the eighteen of them, in the order the search walks over them. `S` is the same thing seen as a set.
- `g * m` is the position `g` followed by the move `m`. The order takes some getting used to: mathcomp applies permutations on the right, so `(g * m) f` is `m (g f)`, and the product reads left to right like a sequence of moves rather than right to left like a composition of functions.
- `h g <= d` is the cut, and `(h (g * m)).+1` is the estimate after a move plus one, which is the assumption that one move changes the estimate by at most one.
- `has (fun m => search d' (g * m)) Sseq` tries every move with one fewer move available, and answers as soon as one of them succeeds.

The last line is the contract of the whole development: **if the search returns false, the position is not within d moves.**

That is a theorem, proved once, about any estimate satisfying the two assumptions. Everything else in the development exists either to discharge those two assumptions for the real table, or to make the search fast enough to run.

What the table has to satisfy, and what it does not. `Coord.v` shows that a summary `coord`, the action `act` of a move on summaries, and any table `D` at all give a legal estimate as soon as `D` passes these two checks:

```
Hypothesis D0    : D (coord 1) = 0.
Hypothesis Dstep : forall x m, m \in Sset -> D x <= (D (act x m)).+1.
```

The first says the solved summary has value zero. The second says that applying a move to a summary lowers its value by at most one. Nothing else is required. In particular the table is never proved to hold the true distances; what is proved of it is only that it is a valid under-estimate, which is a strictly weaker property.

That weaker property is not cheaper to check. Exactness is local too: a table holds the true distances exactly when it satisfies the two conditions above and also, at every summary but the solved one, some move lowers its value by exactly one. That is the same single sweep over the table. What the weaker property buys is not speed but freedom, and it is what section 5 spends: the folded table of the last optimisation is **not** the true distance function and would fail an exactness check, yet it passes these two and is therefore just as good a certificate. So it can be produced by any program in any language, and here an OCaml generator writes it out as Rocq source, to be checked afterwards by evaluating the two conditions on every entry. Those checks are large computations in their own right: the second one runs over each of the 2.2 billion summaries and each of the eighteen moves. They are what the *certificate* files do, each ending in its own `Qed`.

Two reductions cut the top of the tree. First, the superflip looks the same from every angle. There are 48 ways of putting a cube back into the space it came from: any of the six faces can be turned to the top, each of them in four positions, which makes twenty-four, and each of those seen in a mirror as well. Relabelling the superflip's stickers by any of the 48 gives the superflip back again. Being unchanged by all 48, it lets the search take the first move to be U or U^2 instead of any of the eighteen. Second, no shortest solution ever turns the same face twice in a row, nor turns two opposite faces in both orders.

Fixing the first two moves then splits the depth-19 search into $2 \times 18 = 36$ searches of depth 17. The second move ranges over all eighteen, including the three that turn the U face again: those are redundant, but they are left in at this level and the redundancy rules do their work inside the search, which keeps the statement of the split as simple as "any first move from `Sroot`, any second move from `moves`". The 36 are packed into **eighteen files**, one per second move, each carrying both first moves, and that is how the work is spread over the cores of a machine: eighteen files, eighteen `Qed`s, nothing shared.

5 What had to be optimised

The first version worked and was far too slow. Closing the gap between "runs" and "finishes" took a series of changes, each one measured before and after.

Counting in unary is the enemy. The numbers used by the mathcomp library are Peano numbers: 5 is literally the successor of the successor of the successor of the successor of the successor of zero, so adding n costs n steps and comparing costs as much again. Rocq also offers machine integers, 63 bits wide with the missing bit going to the garbage collector, and **persistent arrays** of them [8], both of which cost what the hardware costs. That is also how the table is stored: fifteen of its four-bit entries to a machine integer. Measured here: a machine-integer operation takes about 0.05 microseconds and a Peano-number operation about 1 microsecond, and converting between the two costs about 0.07 microseconds *per unit*, so that converting the number 495 costs 36 microseconds all

by itself. Rewriting the inner loop so that indices, table values and the depth comparison never leave machine integers gave:

operation	before	after
reading the summary of a position	32.0 µs	**0.10 µs**
applying a move to a summary	4.60 µs	**0.12 µs**
reading one entry of the packed table	2.99 µs	**0.13 µs**

`and` and `or` are function calls. Rocq evaluates both sides of a boolean test before combining them, so a guard written "either the value is out of range, or the expensive check holds" runs the expensive check on *every* value. Rewritten as a nested `if`, it runs only on the values that get past the guard. Two otherwise identical certificates, one of each shape: **719.7 s against about 80 s**. The same mistake in the search cost a further factor of 27.8 on one guard. Every guard in the development is a nested `if` now.

The search itself, twelve times faster. `Fast.v` contains a chain of seven versions of the same search, each proved equal to the one before, so no trust is transferred. Measured on one piece at depth 14:

version	seconds	what it removes
the original	70.0	
2	39.1	library numbers out of the inner loop
3	16.5	consult the table before rebuilding anything
4	13.4	stop comparing two tables at the first difference
5	9.7	stop consulting the nine tables once one settles it
6	8.0	compute the three views one at a time
7	**5.9**	carry the list of moves, not the rebuilt position
	11.9x	

Four of those six steps are the same mistake in different clothes: work being done that a lazier evaluator would have skipped.

A sharper estimate, nearly for free. Every position is looked at from three angles, itself and its two rotations about a corner axis, and each angle contributes three table lookups. The estimate is the largest of the nine. More work at each node, far fewer nodes.

What costs memory is the text, not the data. A table written as a list of two million integers occupies 877 MB once loaded, although the data itself is 17 MB: the cost is the syntax tree the kernel holds in memory, not the numbers. Written as an array literal instead, the same block loads in 281 MB, and the full table dropped from 21.5 GB to **5 GB**, which is what allowed nine parallel workers instead of two on a 62 GB machine.

Folding the table by symmetry. Sixteen of the cube's 48 symmetries leave the structure of the summary intact, so summaries come in families of about sixteen, and only one member of each family needs a stored distance: **64 430 families instead of 1 013 760, a factor of 15.73.** A lookup first rewrites the summary into the family's representative. This costs the search , measured at 1.61 times slower at depth 16, and pays everywhere else: a search worker drops from 4.15 GB to **0.85 GB**, so all eighteen pieces now run at once instead of in two waves, and checking the table drops from about 5.4 processor hours to **1.35**. That the fold is legitimate is itself proved. It would not even be needed for correctness: the estimate is never claimed to be the true distance, only to satisfy the two local conditions.

What remains. Against the OCaml program running the same search, Rocq needs about 165 microseconds per position against 0.79, a factor of **209**. That factor, not the algorithm, is why the run takes hours rather than minutes.

6 The files

Forty-six hand-written Rocq files. What each group does.

the cube, as mathematics	
`Cyc.v`	cyclic permutations, built from the list of points they move
`Rubik333.v`	facelets, the six faces, the eighteen moves, the cube group
`Sym.v`, `Sym16.v`	the 48 symmetries of the cube; the 16 used by the fold
`Ball.v`	balls of radius d, and what "the diameter is at most d" means
`Diameter.v`	the superflip, its 20-move sequence, and the final statement

the search, in the abstract	
`Search.v`	the search and its contract: a false answer is a proof
`Coord.v`	any summary plus any checked table gives a legal estimate
`Root.v`	the first move, up to symmetry: U or U^2
`Searchr.v`, `Redun.v`	the rules that forbid redundant move sequences

data structures	
`Table.v`	permutations written as the table of their images
`Tabi.v`	the same on machine integers and arrays, and the bridge between
`ssrint63.v`	the machine-integer toolbox used throughout

the summaries and their tables	
`Coordfs.v`, `Coordfsi.v`	the edge-flip and slice summary, packed into 24 bits
`Fstab.v`, `FsTable.v`, `Fsparity.v`	its table and the checks it must pass
`Phase1.v`	the phase 1 estimate, its table and its certificate, 2215 lines
`Moves.v`	the eighteen moves and the superflip, as tables

the search on the real data	
`Farp1.v`	the three viewing angles and the search built on them, 1404 lines
`Far.v`	the assembly: the superflip is not within d moves, 1124 lines
`Fast.v`, `FastP.v`	the fast search, and the proof that it is the same search
`Runp1_00.v` .. `_17.v`	the eighteen pieces, written by a script
`Farp1main.v`	the theorem over *any* table: 8 seconds, and no data at all
`Farp1inst.v`	the same at the real table and the eighteen real runs

The **certificates** are the files that run the checks, each with its own `Qed`: three for the move and distance tables, one for the second summary table, two for the main table split sixteen ways, and nine more for the folded table. Three of the last group hold the actual numbers and are deliberately kept out of the project file, since they do not exist until the generator has run.

Outside Rocq, `ocaml/rubik_par.ml` and `ocaml/rubik_lb.ml` are the reference implementation. The Rocq search reproduces their node counts exactly, which is how a discrepancy gets caught early, and `bench/p1gen.ml` generates the tables. The `bench/` directory also keeps the experiments that settled design questions, each with its measurements.

7 The development in figures

hand-written Rocq	**46 files, 14 033 lines**
generated table sources, in the repository	about 156 000 lines
generated table sources, too big to store	about 165 MB of literals
OCaml reference programs and generators	2 749 lines

build and run scripts	1 031 lines

Positions visited, measured, and matching the OCaml program exactly:

search depth	one piece	all 18 pieces
14	42 320	784 572
15	547 580	10 185 576
16	7 100 612	130 430 424
17	91 377 680	about 1.7 billion (arithmetic, at the measured growth of 12.87)

Some build costs on the reference machine, a dual-socket Xeon with 62 GB and twelve physical cores, all measured:

the phase 1 estimate and its theory	41 s
the search file over the real tables	1 min 30
checking the folded table, in 27 slices	9 min, about 1.35 processor hours
the structural checks of the fold	about 101 s
compiling one table block to native code	about 6 min, 9–10 GB

8 The theorem, its cost, and what is left

The statement proved at the top of the chain is

```
Theorem superflip_p1far_real : superflip \notin ball Sset p1depth.
```

In words, the superflip is not within `p1depth` moves of the solved cube, where the depth is set by one script before the run. It has **no hypotheses left**: the six computations it rests on, namely the two summary tables, the three move and distance tables and the eighteen searches, each live in their own file behind their own `Qed`. Asking Rocq what the proof assumes reports only the primitives of its machine-integer and array interface. Nothing in the chain is admitted.

At depth 19 this says precisely that the superflip cannot be solved in 19 moves, which is **God's number $\geq$ 20**.

The runs, measured, eighteen pieces on nine workers:

radius	search depth	wall clock	processor time
17	15	13 min 05	35 min 29
18	16	54 min 37	6 h 57
19	**17**	**11 h 13 min**	**85 h 11**

Memory. A worker holds the loaded table and nothing else that grows: the search walks down and back up, so only the current sequence of moves is kept. Measured at **4.15 GB** per worker, identical to three decimals across the nine and the same at every depth, and **0.85 GB** since the table was folded, which is what lets all eighteen pieces run at once. The 19-move run above was made before the fold.

What is left. Two things, stated plainly.

1. The 19-move run predates the fold becoming the official table check. Through the fold the chain has been run at 16 and at 18; running it at 19 is projected at about 13 hours of wall clock, by multiplying the measured 18 figure by the measured growth of 12.87. That same method predicted the earlier run to within 4 %.

2. `Diameter.v` still contains the sentence "the superflip is not within 19 moves" as an admitted placeholder, because that file sits at the bottom of the chain and cannot refer to the search that sits at the top. Both ends exist; a short file at the top has to join them.

And the other half of God's number, that 20 moves always suffice, is not proved here. `Diameter.v` does contain the reduction for it, stated in terms of exactly what an exhaustive search would have to supply: that the 55.9 million families cover every case up to symmetry, and that each of them is solvable in 20 moves. That computation is several orders of magnitude larger than the one this note describes.

References

1. Rokicki, T., Kociemba, H., Davidson, M., Dethridge, J.: The Diameter of the Rubik's Cube Group Is Twenty. SIAM Journal on Discrete Mathematics. 27, 1082–1105 (2013). https://doi.org/10.1137/120867366
2. The Rocq Development Team: The Rocq Prover, https://rocq-prover.org/
3. Mahboubi, A., Tassi, E.: Mathematical Components. Zenodo (2021)
4. Korf, R.E.: Depth-first Iterative-deepening: An Optimal Admissible Tree Search. Artificial Intelligence. 27, 97–109 (1985). https://doi.org/10.1016/0004-3702(85)90084-0
5. Culberson, J.C., Schaeffer, J.: Pattern Databases. Computational Intelligence. 14, 318–334 (1998). https://doi.org/10.1111/0824-7935.00065
6. Korf, R.E.: Finding Optimal Solutions to Rubik's Cube Using Pattern Databases. In: Proceedings of the Fourteenth National Conference on Artificial Intelligence (AAAI-97). pp. 700–705 (1997)
7. Kociemba, H.: The Two-Phase Algorithm, https://kociemba.org/cube.htm
8. Armand, M., Grégoire, B., Spiwack, A., Théry, L.: Extending Coq with Imperative Features and its Application to SAT Verification. In: Interactive Theorem Proving (ITP 2010). pp. 83–98. Springer (2010)